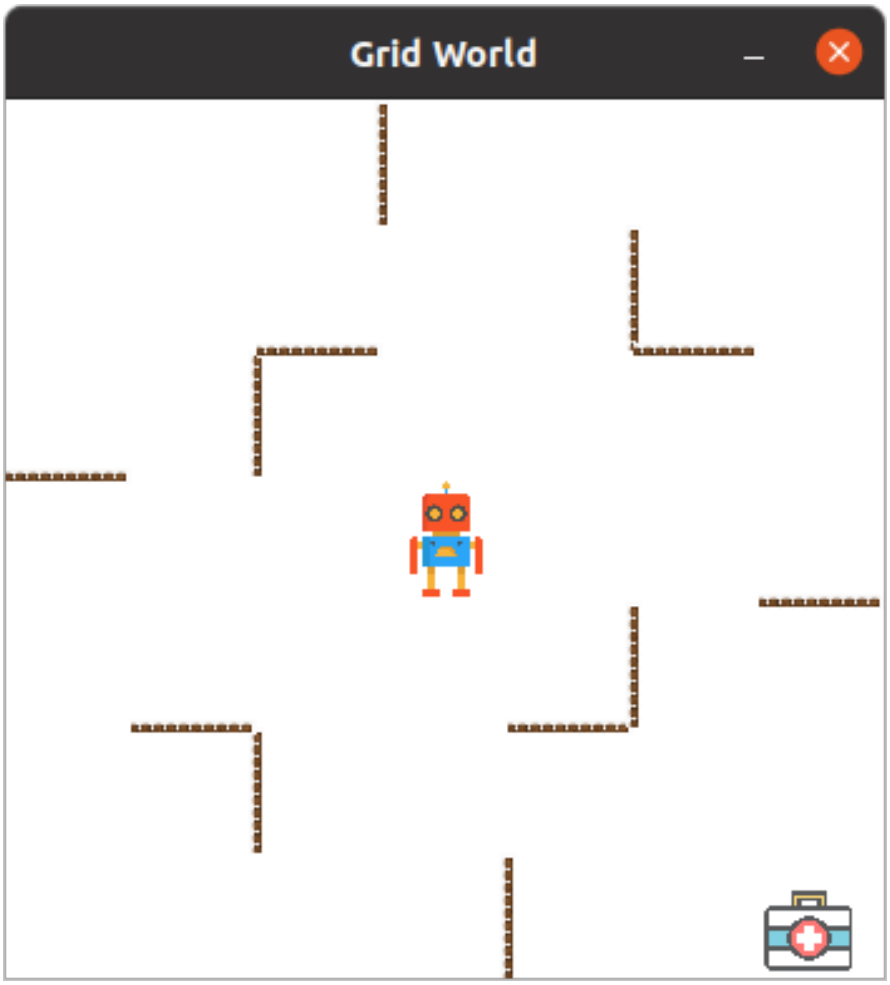
COMP/ELEC 440/557, Final Exam, Part 3



In Part 3, you will program a simple robot to assist in a simplified version of rescue operations following a disaster response. Specifically, the robot will operate in the Grid World shown above, navigating obstacles (walls) and rescuing a medical kit (also referred to as an “object”) by picking it up. We define this environment and task using a Markov Decision Process (MDP) called RescueSimple, which you will first solve using value iteration, then using Q-learning.

The MDP state includes the robot's location. The robot’s action space includes moving up, down, left, right, staying in the current location, or picking up an object. The environment is stochastic, meaning the outcome of the robot’s actions depends on the transition probabilities. The robot receives a reward of -1 for each step taken, and +10 for each object picked up. The discount factor is set as 0.9.

Code: The code for this problem consists of several Python files. You can download all the necessary code and supporting files from the zip file, **P3_P4_starter_code.zip**, available on Canvas.

Files to Edit and Submit: You will fill in portions of **vi_agent.py** and **ql_agent.py** during the assignment. Please *do not* change the other files in this distribution. You will submit **vi_agent.py** and **ql_agent.py** to **Final Exam: Part 3** on Gradescope.

Evaluation: Your code will be autograded for technical correctness and efficiency.

- Please do not change the names of any provided functions or classes within the code.
- Please ensure that your code is compatible with Python 3.10.
- You may only import the numpy package, packages from the Python Standard Library (https://docs.python.org/3/library/index.html), and packages provided in the starter code. Importing any additional packages (such as pytorch, scikit-learn) is not allowed.
- To receive credit, please ensure that your code terminates within 30 minutes when evaluated using the Gradescope autograder. Efficient solutions take less than 5 minutes.
- Use of AI tools (e.g., ChatGPT, Gemini, Claude, Copilot) is not permitted.

Collaboration Policy: Collaborating on the final exam is not permitted.

Citation Policy: Please refer to the course syllabus.

Late Policy: Your answers to the final exam must be submitted on time. Auto-extensions cannot be used for the final exam. Please note that no credit will be given for late submissions.

Installation

For the following questions, you will need to install the following libraries:

- numpy: installation instructions
- pygame: installation instructions
- tqdm: installation instructions

Once they are installed, you can run the **RescueSimple** environment with the following command:

```
# Simple version
python run.py --agent manual --env simple --gui
```

You can manually control the agent using the **arrow keys** (each directional move), the **enter key** (stay), and the **space key** (pick up).

Q1 (30 points). Value Iteration

In this question, you will implement the value iteration algorithm to arrive at the optimal policy for the robot operating in the **RescueSimple** domain. Your task will be to complete the implementation of the **ValueIterationAgent** class in **agent_vi.py**.

Please make sure to review the **StateRescue** class in **models.py**. In the RescueSimple environment, **self.obj_state** is set to None, so the object locations are not directly accessible.

Once implemented, you can check the performance of the value iteration agent with the following commands:

```
# with graphics
python run.py --agent vi --env simple --gui

# without graphics
python run.py --agent vi --env simple --n_episodes 10
```

Your score will be determined based on the accuracy of your learned value function in the RescueSimple domain. (Hint: Please ensure that the learned V-values are sufficiently close to the converged V-values, with an error of less than 0.001 for each state.)

Q2 (20 points). Q-Learning

In this question, you will implement Q-learning to arrive at the optimal policy of RescueSimple. Your task will be to complete the implementation of the **QLearningAgent** class in **agent_ql.py**.

Once implemented, you can check the performance of the Q-learning agent with the following commands:

```
# with graphics
python run.py --agent ql --env simple --gui

# without graphics
python run.py --agent ql --env simple --n_episodes 10
```

Your score will be determined based on the performance of your learned policy in the RescueSimple domain.

Q4.1 (40 points). Custom Agent

In this question, you will implement the policy for the robot operating in the **RescueComplex** domain. You may use any AI method or combine multiple methods. Fill in your solution in the **CustomAgent** class in **agent_custom.py**.

Once implemented, you can check the performance of the custom agent with the following commands:

```
# with graphics
python run.py --agent ca --env complex --n_objs 3 --gui

# without graphics
python run.py --agent ca --env complex --n_objs 3 --n_episodes 10
```

Your score will be determined based on the performance of your policy in the RescueComplex domain. (Hint: When the state space is large, finding the optimal action for every individual state can be highly inefficient. In such cases, approximate methods are often the only practical solution. Although they may not guarantee optimality, these methods typically yield solutions that are near-optimal.)